

DATA STRUCTURES

LINKED LIST, STACK AND QUEUE USING LINKED LIST

1. LINKED LIST

1.1 Definition

A **Linked List** is a dynamic linear data structure in which elements are stored in separate memory locations called **nodes**.

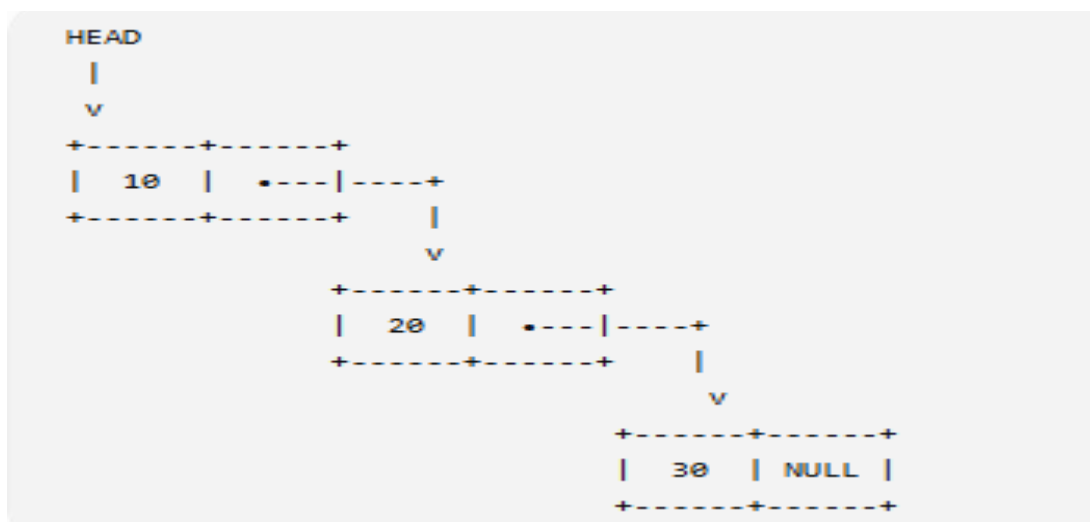
Each node contains two parts:

1. **Data** – stores the actual value.
2. **Next** – stores the address of the next node.

Basic Structure of a Node

```
+-----+-----+  
| DATA | NEXT  |  
+-----+-----+
```

Example



The last node contains NULL because there is no next node.

2. Advantages of Linked List

- Dynamic size
- Memory is allocated when required
- Easy insertion and deletion
- No need for continuous memory locations
- Useful for implementing stacks and queues

3. Limitations of Linked List

- Extra memory is required for storing addresses
 - Direct access using index is not possible
 - Searching takes more time
 - Pointer handling makes the program slightly complex
-

4. NODE DECLARATION IN C

```
struct Node
{
    int data;
    struct Node *next;
};
```

Here:

data → stores the value

next → stores the address of the next node

5. SINGLY LINKED LIST OPERATIONS

The following operations are commonly performed:

1. Insertion at beginning
2. Insertion at end
3. Insertion at any position
4. Insertion after a given value

5. Deletion from beginning
 6. Deletion from end
 7. Deletion from any position
 8. Deletion by value
 9. Display
 10. Searching
-

6. INSERTION AT BEGINNING

Logic

A new node is created and connected before the current first node.

Diagram

Before:

HEAD

|

v

10 -> 20 -> 30 -> NULL

Insert 5:

HEAD

|

v

5 -> 10 -> 20 -> 30 -> NULL

C Function

```
struct Node* insertBeginning(struct Node *head, int value)
{
    struct Node *newNode;

    newNode = (struct Node*)malloc(sizeof(struct Node));

    newNode->data = value;
    newNode->next = head;
```

```
    head = newNode;  
  
    return head;  
}
```

Time Complexity

O(1)

7. INSERTION AT END

Logic

Move to the last node and attach the new node after it.

Diagram

Before:

10 -> 20 -> 30 -> NULL

Insert 40:

10 -> 20 -> 30 -> 40 -> NULL

C Function

```
struct Node* insertEnd(struct Node *head, int value)  
{  
    struct Node *newNode;  
    struct Node *temp;  
  
    newNode = (struct Node*)malloc(sizeof(struct Node));  
  
    newNode->data = value;  
    newNode->next = NULL;  
  
    if (head == NULL)
```

```

{
    return newNode;
}

temp = head;

while (temp->next != NULL)
{
    temp = temp->next;
}

temp->next = newNode;

return head;
}

```

Time Complexity

O(n)

8. INSERTION AT ANY POSITION

Positions are considered from **1**.

Example:

Insert 25 at position 3.

Before:

10 -> 20 -> 30 -> 40 -> NULL

After:

10 -> 20 -> 25 -> 30 -> 40 -> NULL

C Function

struct Node* insertPosition(struct Node *head,

```

        int value, int position)
{
    struct Node *newNode;
    struct Node *temp;
    int i;

    if (position == 1)
    {
        return insertBeginning(head, value);
    }

    newNode = (struct Node*)malloc(sizeof(struct Node));

    newNode->data = value;

    temp = head;

    for (i = 1; i < position - 1; i++)
    {
        if (temp == NULL)
        {
            printf("Invalid position\n");
            free(newNode);
            return head;
        }

        temp = temp->next;
    }

    if (temp == NULL)
    {
        printf("Invalid position\n");
        free(newNode);
        return head;
    }

    newNode->next = temp->next;
    temp->next = newNode;

    return head;
}

```

}

Time Complexity

O(n)

9. INSERTION AFTER A GIVEN VALUE

Suppose the list is:

10 -> 20 -> 30 -> 40 -> NULL

Insert 25 after 20.

Result:

10 -> 20 -> 25 -> 30 -> 40 -> NULL

C Function

```
struct Node* insertAfterValue(struct Node *head,
                             int key, int value)
{
    struct Node *temp;
    struct Node *newNode;

    temp = head;

    while (temp != NULL)
    {
        if (temp->data == key)
        {
            newNode = (struct Node*)malloc(sizeof(struct Node));

            newNode->data = value;
            newNode->next = temp->next;

            temp->next = newNode;
        }
    }
}
```

```

        return head;
    }

    temp = temp->next;
}

printf("Value not found\n");

return head;
}

```

10. DELETION FROM BEGINNING

The first node is removed and head is moved to the second node.

Diagram

Before:

```

HEAD
|
v
10 -> 20 -> 30 -> NULL

```

After deletion:

```

HEAD
|
v
20 -> 30 -> NULL

```

C Function

```

struct Node* deleteBeginning(struct Node *head)
{
    struct Node *temp;

    if (head == NULL)
    {
        printf("List is empty\n");
    }
}

```

```

        return head;
    }

    temp = head;

    head = head->next;

    free(temp);

    return head;
}

```

Time Complexity

O(1)

11. DELETION FROM END

The last node is removed.

Diagram

Before:

10 -> 20 -> 30 -> NULL

After:

10 -> 20 -> NULL

C Function

```

struct Node* deleteEnd(struct Node *head)
{
    struct Node *temp;

    if (head == NULL)
    {
        printf("List is empty\n");
    }
}

```

```

        return head;
    }

    if (head->next == NULL)
    {
        free(head);
        return NULL;
    }

    temp = head;

    while (temp->next->next != NULL)
    {
        temp = temp->next;
    }

    free(temp->next);

    temp->next = NULL;

    return head;
}

```

Time Complexity

O(n)

12. DELETION FROM ANY POSITION

Example:

Delete position 2.

Before:

10 -> 20 -> 30 -> 40 -> NULL

After:

10 -> 30 -> 40 -> NULL

C Function

```
struct Node* deletePosition(struct Node *head, int position)
{
    struct Node *temp;
    struct Node *deleteNode;
    int i;

    if (head == NULL)
    {
        printf("List is empty\n");
        return head;
    }

    if (position == 1)
    {
        return deleteBeginning(head);
    }

    temp = head;

    for (i = 1; i < position - 1; i++)
    {
        if (temp == NULL || temp->next == NULL)
        {
            printf("Invalid position\n");
            return head;
        }

        temp = temp->next;
    }

    if (temp->next == NULL)
    {
        printf("Invalid position\n");
        return head;
    }
}
```

```
deleteNode = temp->next;

temp->next = deleteNode->next;

free(deleteNode);

return head;
}
```

13. DELETION BY VALUE

This operation searches for a value and deletes its **first occurrence**.

Example:

10 -> 20 -> 30 -> 40 -> NULL

Delete 30.

Result:

10 -> 20 -> 40 -> NULL

C Function

```
struct Node* deleteValue(struct Node *head, int value)
{
    struct Node *temp;
    struct Node *deleteNode;

    if (head == NULL)
    {
        printf("List is empty\n");
        return head;
    }

    if (head->data == value)
    {
        return deleteBeginning(head);
    }
}
```

```

temp = head;

while (temp->next != NULL)
{
    if (temp->next->data == value)
    {
        deleteNode = temp->next;

        temp->next = deleteNode->next;

        free(deleteNode);

        return head;
    }

    temp = temp->next;
}

printf("Value not found\n");

return head;
}

```

14. DISPLAY LINKED LIST

C Function

```

void display(struct Node *head)
{
    struct Node *temp;

    if (head == NULL)
    {
        printf("List is empty\n");
        return;
    }

    temp = head;

```

```

printf("HEAD -> ");

while (temp != NULL)
{
    printf("%d -> ", temp->data);
    temp = temp->next;
}

printf("NULL\n");
}

```

Example Output

HEAD -> 10 -> 20 -> 30 -> 40 -> NULL

15. SEARCHING IN LINKED LIST

C Function

```

void search(struct Node *head, int value)
{
    int position = 1;

    while (head != NULL)
    {
        if (head->data == value)
        {
            printf("Value found at position %d\n",
                position);
            return;
        }

        head = head->next;
        position++;
    }

    printf("Value not found\n");
}

```

Time Complexity

O(n)

16. COMPLETE SINGLY LINKED LIST PROGRAM

```
#include <stdio.h>
#include <stdlib.h>

struct Node
{
    int data;
    struct Node *next;
};

struct Node* insertBeginning(struct Node *head, int value)
{
    struct Node *newNode;

    newNode = (struct Node*)malloc(sizeof(struct Node));

    newNode->data = value;
    newNode->next = head;

    head = newNode;

    return head;
}

struct Node* insertEnd(struct Node *head, int value)
{
    struct Node *newNode;
    struct Node *temp;

    newNode = (struct Node*)malloc(sizeof(struct Node));

    newNode->data = value;
    newNode->next = NULL;
```

```

    if (head == NULL)
        return newNode;

    temp = head;

    while (temp->next != NULL)
        temp = temp->next;

    temp->next = newNode;

    return head;
}

struct Node* insertPosition(struct Node *head,
                           int value, int position)
{
    struct Node *newNode;
    struct Node *temp;
    int i;

    if (position == 1)
        return insertBeginning(head, value);

    newNode = (struct Node*)malloc(sizeof(struct Node));

    newNode->data = value;

    temp = head;

    for (i = 1; i < position - 1; i++)
    {
        if (temp == NULL)
        {
            printf("Invalid position\n");
            free(newNode);
            return head;
        }

        temp = temp->next;
    }

```

```

    }

    if (temp == NULL)
    {
        printf("Invalid position\n");
        free(newNode);
        return head;
    }

    newNode->next = temp->next;
    temp->next = newNode;

    return head;
}

struct Node* insertAfterValue(struct Node *head,
                             int key, int value)
{
    struct Node *temp;
    struct Node *newNode;

    temp = head;

    while (temp != NULL)
    {
        if (temp->data == key)
        {
            newNode = (struct Node*)malloc(sizeof(struct Node));

            newNode->data = value;
            newNode->next = temp->next;

            temp->next = newNode;

            return head;
        }

        temp = temp->next;
    }
}

```

```

    printf("Value not found\n");

    return head;
}

struct Node* deleteBeginning(struct Node *head)
{
    struct Node *temp;

    if (head == NULL)
    {
        printf("List is empty\n");
        return head;
    }

    temp = head;
    head = head->next;

    free(temp);

    return head;
}

struct Node* deleteEnd(struct Node *head)
{
    struct Node *temp;

    if (head == NULL)
    {
        printf("List is empty\n");
        return head;
    }

    if (head->next == NULL)
    {
        free(head);
        return NULL;
    }

    temp = head;

```

```

while (temp->next->next != NULL)
    temp = temp->next;

free(temp->next);
temp->next = NULL;

return head;
}

struct Node* deletePosition(struct Node *head, int position)
{
    struct Node *temp;
    struct Node *deleteNode;
    int i;

    if (head == NULL)
    {
        printf("List is empty\n");
        return head;
    }

    if (position == 1)
        return deleteBeginning(head);

    temp = head;

    for (i = 1; i < position - 1; i++)
    {
        if (temp == NULL || temp->next == NULL)
        {
            printf("Invalid position\n");
            return head;
        }

        temp = temp->next;
    }

    if (temp->next == NULL)
    {

```

```

        printf("Invalid position\n");
        return head;
    }

    deleteNode = temp->next;
    temp->next = deleteNode->next;

    free(deleteNode);

    return head;
}

struct Node* deleteValue(struct Node *head, int value)
{
    struct Node *temp;
    struct Node *deleteNode;

    if (head == NULL)
    {
        printf("List is empty\n");
        return head;
    }

    if (head->data == value)
        return deleteBeginning(head);

    temp = head;

    while (temp->next != NULL)
    {
        if (temp->next->data == value)
        {
            deleteNode = temp->next;
            temp->next = deleteNode->next;

            free(deleteNode);

            return head;
        }
    }
}

```

```

        temp = temp->next;
    }

    printf("Value not found\n");

    return head;
}

void display(struct Node *head)
{
    struct Node *temp = head;

    if (head == NULL)
    {
        printf("List is empty\n");
        return;
    }

    printf("HEAD -> ");

    while (temp != NULL)
    {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }

    printf("NULL\n");
}

void search(struct Node *head, int value)
{
    int position = 1;

    while (head != NULL)
    {
        if (head->data == value)
        {
            printf("Value found at position %d\n",
                position);
            return;
        }
    }
}

```

```

    }

    head = head->next;
    position++;
}

printf("Value not found\n");
}

int main()
{
    struct Node *head = NULL;

    head = insertBeginning(head, 20);
    head = insertBeginning(head, 10);

    head = insertEnd(head, 40);

    head = insertPosition(head, 30, 3);

    head = insertAfterValue(head, 30, 35);

    printf("Linked List:\n");
    display(head);

    head = deleteBeginning(head);
    printf("\nAfter deleting beginning:\n");
    display(head);

    head = deleteEnd(head);
    printf("\nAfter deleting end:\n");
    display(head);

    head = deletePosition(head, 2);
    printf("\nAfter deleting position 2:\n");
    display(head);

    head = deleteValue(head, 30);
    printf("\nAfter deleting value 30:\n");
    display(head);
}

```

```

printf("\nSearching 20:\n");
search(head, 20);

return 0;
}

```

STACK USING LINKED LIST

Definition

A stack is a linear data structure that follows the LIFO principle.

LIFO means:

Last In → First Out

Example:

If we insert:

10 → 20 → 30

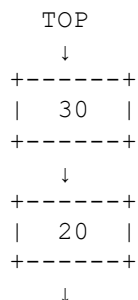
Then deletion happens as:

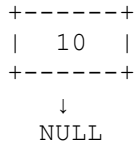
30 → 20 → 10

Basic Operations

1. Push – Insert an element into the stack.
2. Pop – Delete the top element from the stack.
3. Peek – Display the top element without deleting it.
4. Display – Display all elements of the stack.

Representation





Stack Implementation using LL

```

#include <stdio.h>
#include <stdlib.h>

struct Node
{
    int data;
    struct Node *next;
};

struct Node *top = NULL;

/* Push Operation */
void push()
{
    struct Node *newNode;

    newNode = (struct Node *)malloc(sizeof(struct Node));

    if (newNode == NULL)
    {
        printf("Stack Overflow\n");
        return;
    }

    printf("Enter value: ");
    scanf("%d", &newNode->data);

    newNode->next = top;
    top = newNode;

    printf("%d pushed into stack.\n", newNode->data);
}

```

```

/* Pop Operation */
void pop()
{
    struct Node *temp;

    if (top == NULL)
    {
        printf("Stack Underflow\n");
        return;
    }

    temp = top;

    printf("%d popped from stack.\n", top->data);

    top = top->next;

    free(temp);
}

/* Peek Operation */
void peek()
{
    if (top == NULL)
    {
        printf("Stack is empty.\n");
        return;
    }

    printf("Top element: %d\n", top->data);
}

/* Display Operation */
void display()
{
    struct Node *temp;

    if (top == NULL)
    {

```

```

    printf("Stack is empty.\n");
    return;
}

temp = top;

printf("Stack elements:\n");

while (temp != NULL)
{
    printf("%d\n", temp->data);
    temp = temp->next;
}

}

/* Main Function */
int main()
{
    int choice;

    while (1)
    {
        printf("\n===== STACK USING LINKED LIST =====\n");
        printf("1. Push\n");
        printf("2. Pop\n");
        printf("3. Peek\n");
        printf("4. Display\n");
        printf("5. Exit\n");

        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1:
                push();
                break;

            case 2:

```

```

        pop();
        break;

    case 3:
        peek();
        break;

    case 4:
        display();
        break;

    case 5:
        printf("Program terminated.\n");
        exit(0);

    default:
        printf("Invalid choice.\n");
    }
}

return 0;

}

```

How Push Works

Suppose the stack contains:

```

TOP
↓
30
↓
20
↓
10

```

↓
NULL

Now we push 40.

A new node is created:

40

Then:

`newNode->next = top;`

This connects 40 with the previous top.

Finally:

`top = newNode;`

The new stack becomes:

TOP
↓
40
↓
30
↓
20
↓
10
↓
NULL

How Pop Works

Suppose:

TOP
↓
40
↓
30
↓
20
↓
NULL

During pop:

1. Store the top node in temp.
2. Move top to the next node.
3. Delete the old top node using free().

After pop:

TOP
↓
30
↓
20
↓
NULL

Overflow and Underflow

Stack Overflow:

In a linked-list stack, overflow occurs when memory cannot be allocated.

if (newNode == NULL)

Stack Underflow:

Underflow occurs when we try to pop from an empty stack.

if (top == NULL)

Important Points

- Push always happens at the TOP.
- Pop always happens from the TOP.
- Peek shows the TOP element without deleting it.
- Linked-list stack does not require a fixed size.
- malloc() is used to create a new node.
- free() is used to release memory after deletion.
- Stack follows LIFO: Last In, First Out.

21. QUEUE USING LINKED LIST

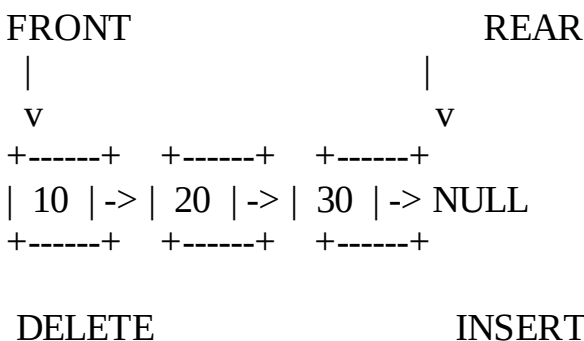
Definition

A **Queue** is a linear data structure that follows:

FIFO – First In, First Out

The element inserted first is removed first.

Diagram



Main Operations

- Enqueue
- Dequeue
- Peek
- Display

Queue using Linked List

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct Node
```

$$\{$$

```
int data;
```

```
struct Node *next;
```

```
};
```

```
struct Node *front = NULL;
```

```
struct Node *rear = NULL;
```

Insert an element into the queue

```
void enqueue()
```

```
{
```

```
    struct Node *newNode;
```

```
    newNode = (struct Node *)malloc(sizeof(struct Node));
```

```
    if (newNode == NULL)
```

```
    {
```

```
        printf("Queue Overflow\n");
```

```
        return;
```

```
    }
```

```
    printf("Enter value: ");
```

```
    scanf("%d", &newNode->data);
```

```
    newNode->next = NULL;
```

```
    /* If queue is empty */
```

```
    if (front == NULL)
```

```
    {
```

```
        front = newNode;
```

```
        rear = newNode;
```

```
    }
```

```

else
{
    rear->next = newNode;
    rear = newNode;
}

printf("%d inserted into queue.\n", newNode->data);
}

```

Delete an element from the queue

```

void dequeue()
{
    struct Node *temp;

    if (front == NULL)
    {
        printf("Queue Underflow\n");
        return;
    }

    temp = front;

    printf("%d deleted from queue.\n", front->data);

    front = front->next;

    /* If queue becomes empty */
    if (front == NULL)
    {

```

```
    rear = NULL;
}

free(temp);
}
```

Display all elements

```
void display()
{
    struct Node *temp;

    if (front == NULL)
    {
        printf("Queue is empty.\n");
        return;
    }

    temp = front;

    printf("Queue: ");

    while (temp != NULL)
    {
        printf("%d ", temp->data);
        temp = temp->next;
    }

    printf("\n");
}
```

Show the front element

```
void peek()
{
    if (front == NULL)
    {
        printf("Queue is empty.\n");
        return;
    }

    printf("Front element: %d\n", front->data);
}

/* Main function */
int main()
{
    int choice;

    while (1)
    {
        printf("\n===== QUEUE USING LINKED LIST =====\n");
        printf("1. Enqueue\n");
        printf("2. Dequeue\n");
        printf("3. Peek\n");
        printf("4. Display\n");
        printf("5. Exit\n");

        printf("Enter your choice: ");
        scanf("%d", &choice);
```

```
switch (choice)
{
    case 1:
        enqueue();
        break;

    case 2:
        dequeue();
        break;

    case 3:
        peek();
        break;

    case 4:
        display();
        break;

    case 5:
        printf("Program terminated.\n");
        exit(0);

    default:
        printf("Invalid choice.\n");
}

return 0;
}}
```

Implement the Doubly Linked list

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct Node
```

```
{
```

```
    int data;
```

```
    struct Node *prev;
```

```
    struct Node *next;
```

```
};
```

```
struct Node *head = NULL;
```

Insert at Beginning

```
void insertAtBeginning(int value)
```

```
{
```

```
    struct Node *newNode;
```

```
    newNode = (struct Node *)malloc(sizeof(struct Node));
```

```
    newNode->data = value;
```

```
    newNode->prev = NULL;
```

```
    newNode->next = head;
```

```
    if (head != NULL)
```

```
    {
```

```
        head->prev = newNode;
```

```
    }
```

```
    head = newNode;
```

```
}
```

Insert at End

```
void insertAtEnd(int value)
```

```
{
```

```
    struct Node *newNode;
```

```
    struct Node *temp;
```

```
    newNode = (struct Node *)malloc(sizeof(struct Node));
```

```
    newNode->data = value;
```

```
    newNode->next = NULL;
```

```
    if (head == NULL)
```

```
    {
```

```
        newNode->prev = NULL;
```

```
        head = newNode;
```

```
        return;
```

```
    }
```

```
    temp = head;
```

```
    while (temp->next != NULL)
```

```
    {
```

```
        temp = temp->next;
```

```
    }
```

```
    newNode->prev = temp;
```

```
    temp->next = newNode;
```

```
}
```

Insert at Any Position

```
void insertAtPosition(int value, int position)
{
    struct Node *newNode;
    struct Node *temp;
    int i;

    if (position == 1)
    {
        insertAtBeginning(value);
        return;
    }

    newNode = (struct Node *)malloc(sizeof(struct Node));

    newNode->data = value;

    temp = head;

    for (i = 1; i < position - 1; i++)
    {
        if (temp == NULL)
        {
            printf("Invalid position\n");
            free(newNode);
            return;
        }

        temp = temp->next;
```

```

    }

    if (temp == NULL)
    {
        printf("Invalid position\n");
        free(newNode);
        return;
    }

    newNode->next = temp->next;
    newNode->prev = temp;

    if (temp->next != NULL)
    {
        temp->next->prev = newNode;
    }

    temp->next = newNode;
}

```

Delete from Beginning

```

void deleteFromBeginning()
{
    struct Node *temp;

    if (head == NULL)
    {
        printf("List is empty\n");
        return;
    }
}

```

```
temp = head;

head = head->next;

if (head != NULL)
{
    head->prev = NULL;
}

free(temp);
}
```

Delete from End

```
void deleteFromEnd()
{
    struct Node *temp;

    if (head == NULL)
    {
        printf("List is empty\n");
        return;
    }

    temp = head;

    if (temp->next == NULL)
    {
        head = NULL;
        free(temp);
    }
}
```

```

        return;
    }

    while (temp->next != NULL)
    {
        temp = temp->next;
    }

    temp->prev->next = NULL;

    free(temp);
}

```

Delete from Any Position

```

void deleteFromPosition(int position)
{
    struct Node *temp;
    int i;

    if (head == NULL)
    {
        printf("List is empty\n");
        return;
    }

    if (position == 1)
    {
        deleteFromBeginning();
        return;
    }
}

```

```
temp = head;

for (i = 1; i < position; i++)
{
    if (temp == NULL)
    {
        printf("Invalid position\n");
        return;
    }

    temp = temp->next;
}

if (temp == NULL)
{
    printf("Invalid position\n");
    return;
}

if (temp->next != NULL)
{
    temp->next->prev = temp->prev;
}

temp->prev->next = temp->next;

free(temp);
}
```

Display Forward

```
void displayForward()
{
    struct Node *temp;

    if (head == NULL)
    {
        printf("List is empty\n");
        return;
    }

    temp = head;

    printf("\nForward: ");

    while (temp != NULL)
    {
        printf("%d", temp->data);

        if (temp->next != NULL)
        {
            printf(" <-> ");
        }

        temp = temp->next;
    }

    printf("\n");
}
```

Display Backward

```
void displayBackward()
{
    struct Node *temp;

    if (head == NULL)
    {
        printf("List is empty\n");
        return;
    }

    temp = head;

    while (temp->next != NULL)
    {
        temp = temp->next;
    }

    printf("\nBackward: ");

    while (temp != NULL)
    {
        printf("%d", temp->data);

        if (temp->prev != NULL)
        {
            printf(" <-> ");
        }
    }
}
```

```

        temp = temp->prev;
    }

    printf("\n");
}

int main()
{
    int choice;
    int value;
    int position;

    while (1)
    {
        printf("\n\n===== DOUBLY LINKED LIST =====");
        printf("\n1. Insert at Beginning");
        printf("\n2. Insert at End");
        printf("\n3. Insert at Position");
        printf("\n4. Delete from Beginning");
        printf("\n5. Delete from End");
        printf("\n6. Delete from Position");
        printf("\n7. Display Forward");
        printf("\n8. Display Backward");
        printf("\n9. Exit");

        printf("\n\nEnter your choice: ");
        scanf("%d", &choice);

        switch (choice)

```

```
{  
    case 1:  
        printf("Enter value: ");  
        scanf("%d", &value);  
  
        insertAtBeginning(value);  
        break;  
  
    case 2:  
        printf("Enter value: ");  
        scanf("%d", &value);  
  
        insertAtEnd(value);  
        break;  
  
    case 3:  
        printf("Enter value: ");  
        scanf("%d", &value);  
  
        printf("Enter position: ");  
        scanf("%d", &position);  
  
        insertAtPosition(value, position);  
        break;  
  
    case 4:  
        deleteFromBeginning();  
        break;  
  
    case 5:
```

```
        deleteFromEnd();
        break;

    case 6:
        printf("Enter position: ");
        scanf("%d", &position);

        deleteFromPosition(position);
        break;

    case 7:
        displayForward();
        break;

    case 8:
        displayBackward();
        break;

    case 9:
        printf("Program ended.\n");
        exit(0);

    default:
        printf("Invalid choice\n");
    }
}

return 0;
}
```

Implementation of Sparse Matrix using LINKED LIST

```
#include <stdio.h>
#include <stdlib.h>

struct Node
{
    int row;
    int col;
    int value;
    struct Node *next;
};

struct Node *head = NULL;
```

Check whether matrix is sparse

```
int isSparse(int matrix[][10], int rows, int cols)
{
    int zero = 0;
    int nonZero = 0;

    for (int i = 0; i < rows; i++)
    {
        for (int j = 0; j < cols; j++)
        {
            if (matrix[i][j] == 0)
            {
                zero++;
            }
            else
            {
                nonZero++;
            }
        }
    }
}
```

```

}

printf("\nZero elements = %d", zero);
printf("\nNon-zero elements = %d\n", nonZero);

if (zero > nonZero)
{
    return 1;
}
else
{
    return 0;
}
}

```

Insert non-zero element into linked list

```

void insert(int row, int col, int value)
{
    struct Node *newNode;
    struct Node *temp;

    newNode = (struct Node *)malloc(sizeof(struct Node));

    newNode->row = row;
    newNode->col = col;
    newNode->value = value;
    newNode->next = NULL;

    if (head == NULL)
    {
        head = newNode;
    }
    else
    {

```

```

temp = head;

while (temp->next != NULL)
{
    temp = temp->next;
}

temp->next = newNode;
}
}

```

Display sparse matrix

```

void display()
{
    struct Node *temp;

    temp = head;

    printf("\nRow\tColumn\tValue\n");

    while (temp != NULL)
    {
        printf("%d\t%d\t%d\n",
            temp->row,
            temp->col,
            temp->value);

        temp = temp->next;
    }
}

```

Main function

```

int main()

```

```

{
    int matrix[10][10];
    int rows, cols;

    printf("Enter number of rows: ");
    scanf("%d", &rows);

    printf("Enter number of columns: ");
    scanf("%d", &cols);

    printf("\nEnter matrix elements:\n");

    for (int i = 0; i < rows; i++)
    {
        for (int j = 0; j < cols; j++)
        {
            scanf("%d", &matrix[i][j]);
        }
    }
}

```

Check whether matrix is sparse

```

if (isSparse(matrix, rows, cols))
{
    printf("\nMatrix is Sparse.\n");

    /* Store only non-zero elements */
    for (int i = 0; i < rows; i++)
    {
        for (int j = 0; j < cols; j++)
        {
            if (matrix[i][j] != 0)
            {
                insert(i, j, matrix[i][j]);
            }
        }
    }
}

```

```

    }

    Display linked list
    display();
}
else
{
    printf("\nMatrix is NOT Sparse.\n");
}

return 0;
}

```

LINKED LIST VS STACK VS QUEUE

Structure	Main Rule	Insertion	Deletion
Linked List	Sequential	Beginning/End/Position	Beginning/End/Position
Stack	LIFO	Top	Top
Queue	FIFO	Rear	Front

26. TIME COMPLEXITY

<u>Operation</u>	<u>Time Complexity</u>
Insert at Beginning	O(1)
Insert at End	O(n)
Insert at Position	O(n)
Insert After Value	O(n)
Delete Beginning	O(1)
Delete End	O(n)
Delete Position	O(n)
Delete by Value	O(n)
Search	O(n)
Display	O(n)
Stack Push	O(1)
Stack Pop	O(1)
Stack Peek	O(1)
Queue Enqueue	O(1)
Queue Dequeue	O(1)
Queue Peek	O(1)

27. IMPORTANT POINTS FOR EXAM

1. A linked list uses **dynamic memory allocation**.
2. Each node contains **data and a link**.
3. The first node is accessed using head.
4. The last node of a singly linked list points to NULL.
5. Insertion at the beginning takes **O(1)** time.
6. Searching a linked list takes **O(n)** time.
7. Stack follows **LIFO**.
8. Queue follows **FIFO**.
9. Stack insertion and deletion are performed at the **top**.
10. Queue insertion is performed at the **rear** and deletion at the **front**.
11. `malloc()` is used to allocate memory dynamically.
12. `free()` is used to release dynamically allocated memory.
13. Always check whether the list/stack/queue is empty before deletion.
14. Position numbering in the above programs starts from **1**.
15. The delete-by-value function removes the **first matching value**.

